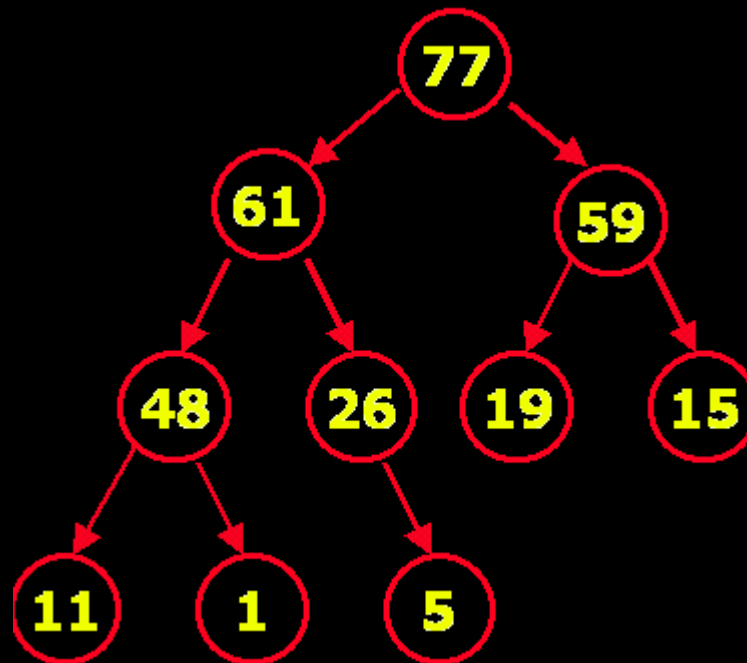


O que é “heap”?

- Definições de **heap**:
 - (**conhecida**) Uma grande quantidade de memória de onde o programador pode alocar ou liberar blocos para uso dentro de um programa.
 - (**nova**) Uma árvore binária balanceada, justificada à esquerda, onde nenhum nó possui um valor maior que seu pai.
- Não possuem nada em comum:
 - Heapsort usa a segunda

A Árvore-Heap

- Uma **árvore-heap** é uma árvore binária justificada à esquerda, organizada por níveis, onde o nodo pai de uma subárvore sempre é maior que todos os seus descendentes.



Origem da Metáfora:

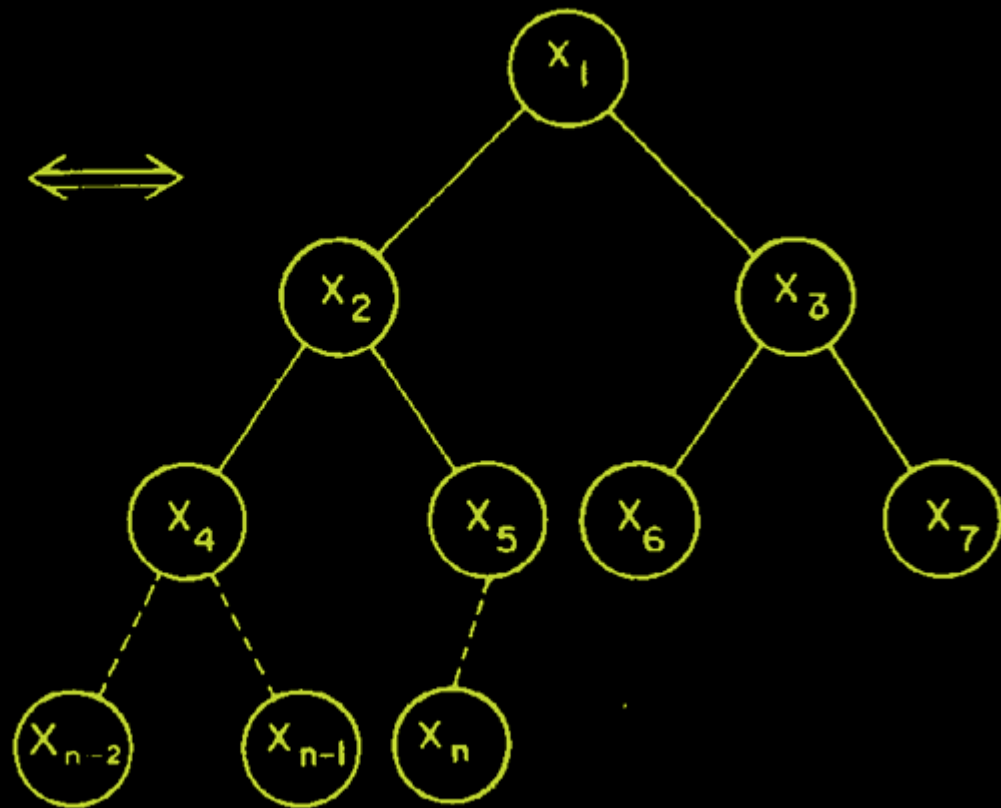
- Efeito Nóz-do-Pará (*Brazil Nut Effect*)
 - Fenômeno da flutuação reversa em material granular



- Veja alguns vídeos...

Visão do arquivo $x_1, \dots, x_n$ por níveis

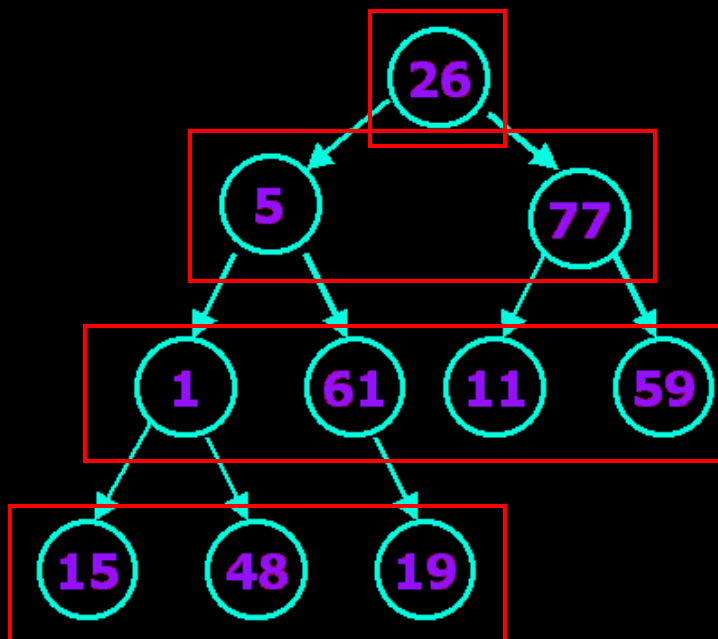
$(x_1, \dots, x_n)$



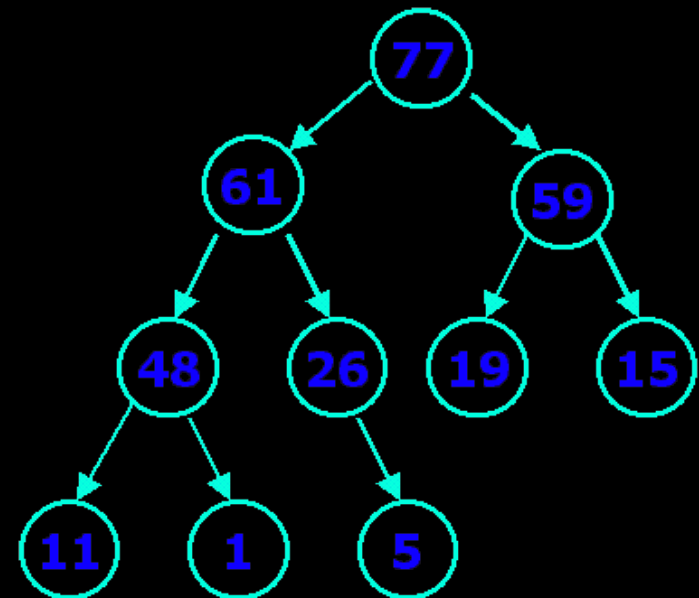
Exemplo

(**26** **5** **77** **1** **61** **11** **59** **15** **48** **19**)

Arquivo original por níveis

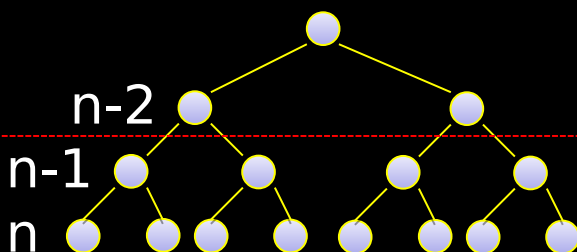


Árvore-Heap resultante

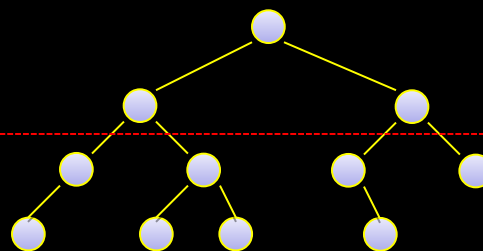


Árvores binárias balanceadas

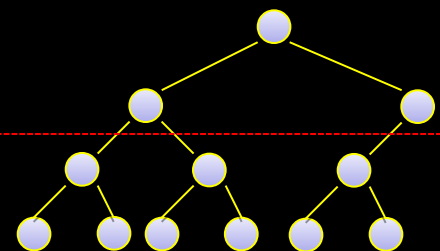
- Lembre:
 - A **profundidade de um nodo** é sua distância da raiz
 - A **profundidade de uma árvore** é a profundidade de seu nodo mais profundo
- Uma árvore binária de profundidade n está balanceada se todos os nodos da profundidade **0** até **$n-2$** possuírem dois filhos



Balanceada



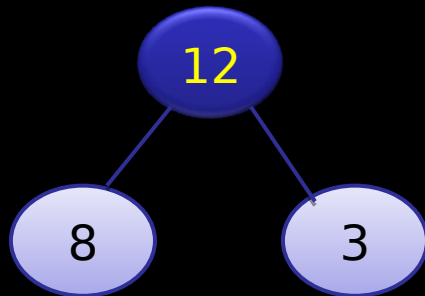
Balanceada



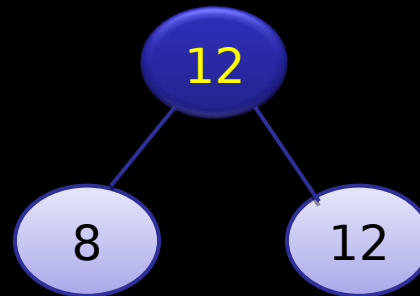
Não balanceada

A propriedade-heap

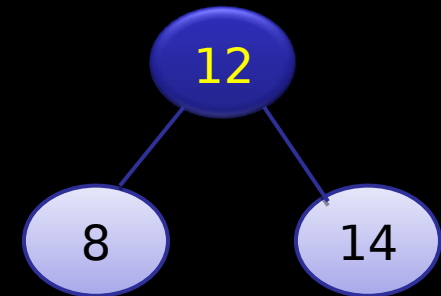
- Um nodo possui a propriedade-heap se nenhum de seus descendentes é maior que ele.



Raiz possui
propriedade-heap



Raiz possui
propriedade-heap

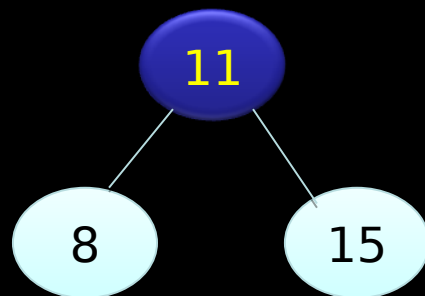


Raiz não possui
propriedade-heap

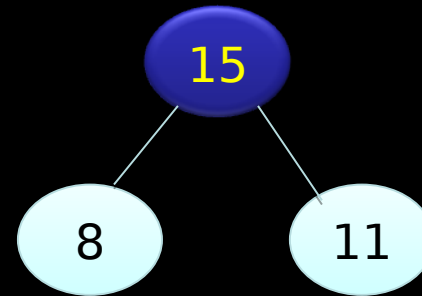
- Todas as folhas possuem propriedade-heap
- Uma árvore binária é um heap se todos os seus nodos possuem a propriedade-heap

Peneirar para cima (**siftUp**)

- Dado um nodo não-heap, ele pode adquirir a propriedade-heap trocando seu valor com seu maior filho.



Nodo azul não possui propriedade-heap



Nodo azul possui propriedade-heap

- Isto é chamado **sifting up**
- Observe que o filho pode ter perdido a propriedade-heap

Construindo um monte I

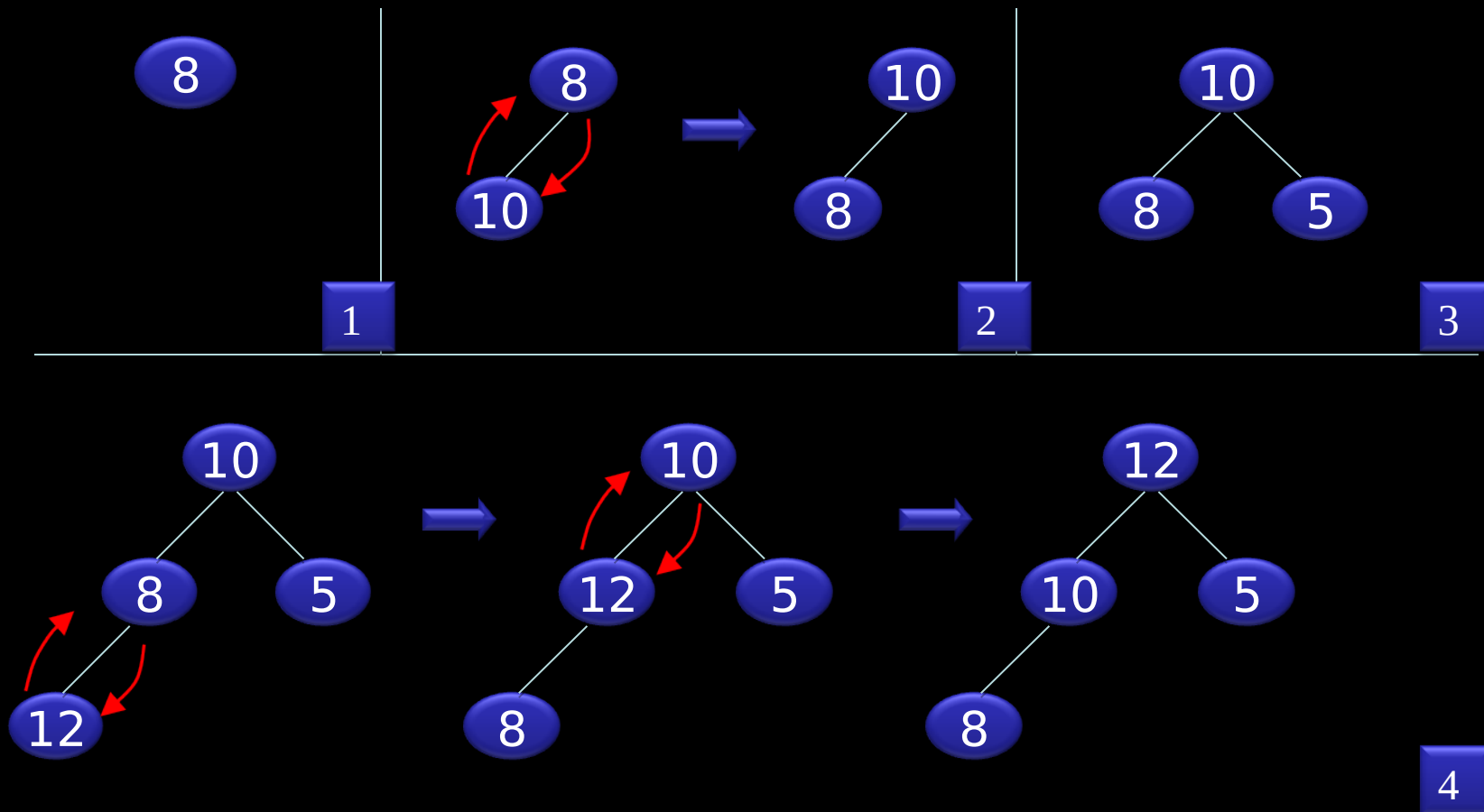
- Uma árvore unitária é um heap
- Construimos um heap adicionando nodos um a um:
 - Adicione o nodo imediatamente à direita do nodo mais à direita no nível mais profundo.
 - Se o nível mais profundo estiver completo, inicie um novo nível
- Exemplos:



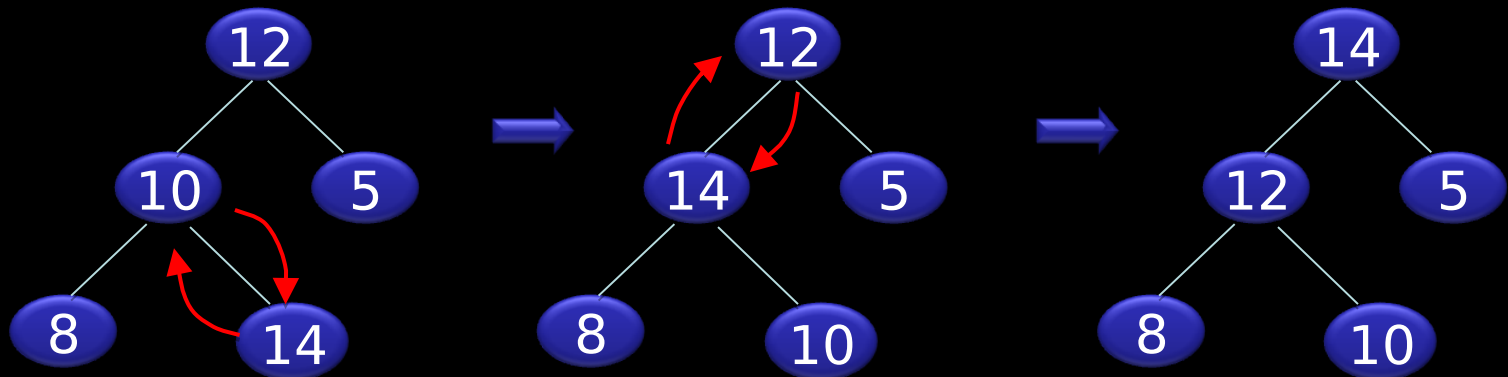
Construindo um monte II

- Toda vez que adicionamos um nodo, poderemos estar destruindo a propriedade-heap de seu pai.
 - Para arrumar isto, peneiramos para cima.
- Como o valor do nodo pai se altera, isto pode alterar a propriedade-heap em seu pai.
- Repetimos o sifting up, subindo na árvore até que
 - ou
 - Atinjamos nodos cujos valores não tenham de ser trocados
 - o pai ainda é maior que o filho
 - ou
 - Atinjamos a raiz

Construindo um monte III



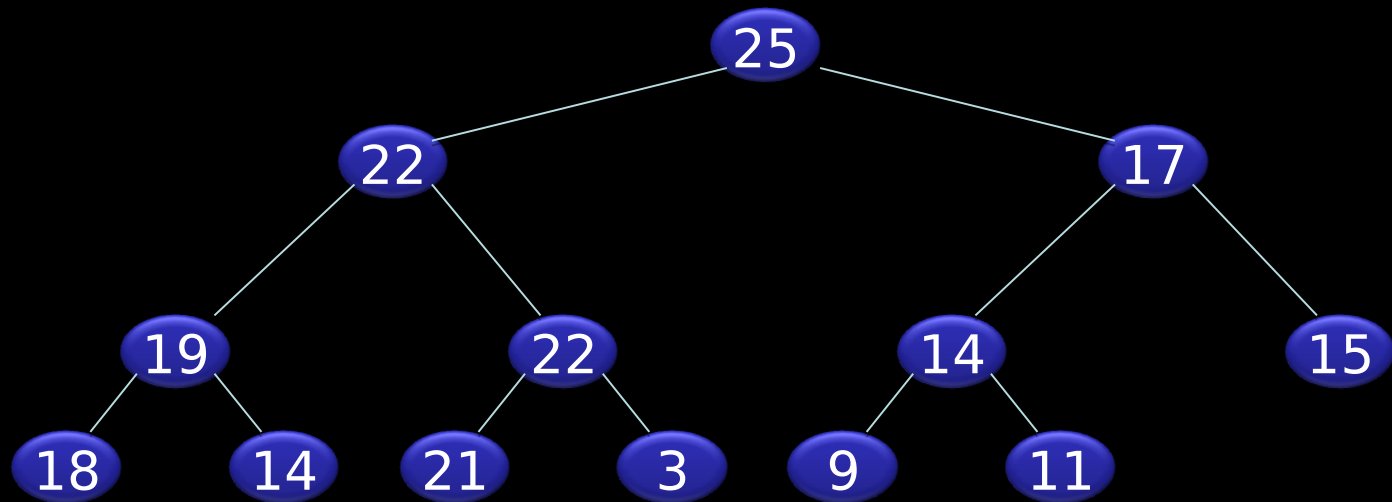
Outros filhos não são afetados



- ✓ O nodo contendo 8 não é afetado porque seu pai fica maior, não menor
- ✓ O nodo contendo 5 não é afetado porque seu pai fica maior, não menor
- ✓ O nodo contendo 8 continua não sendo afetado porque seu pai, embora tenha diminuído, continua sendo maior do que era antes

Um heap-exemplo

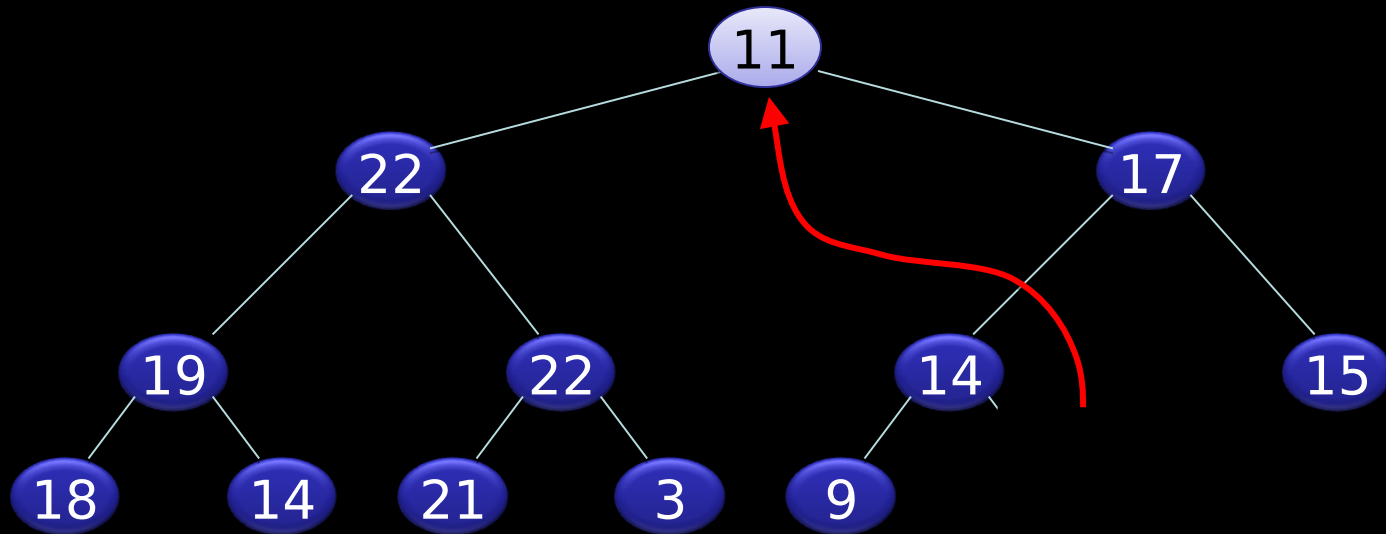
- Aqui uma árvore binária-exemplo depois que ela foi **amontoadada** (*heapificada*...)



- Observe que amontoar **não** significa **ordenar**
- Amontoar também não altera a forma da árvore. Se ela começou justificada à esquerda, continua assim.

Removendo a raiz

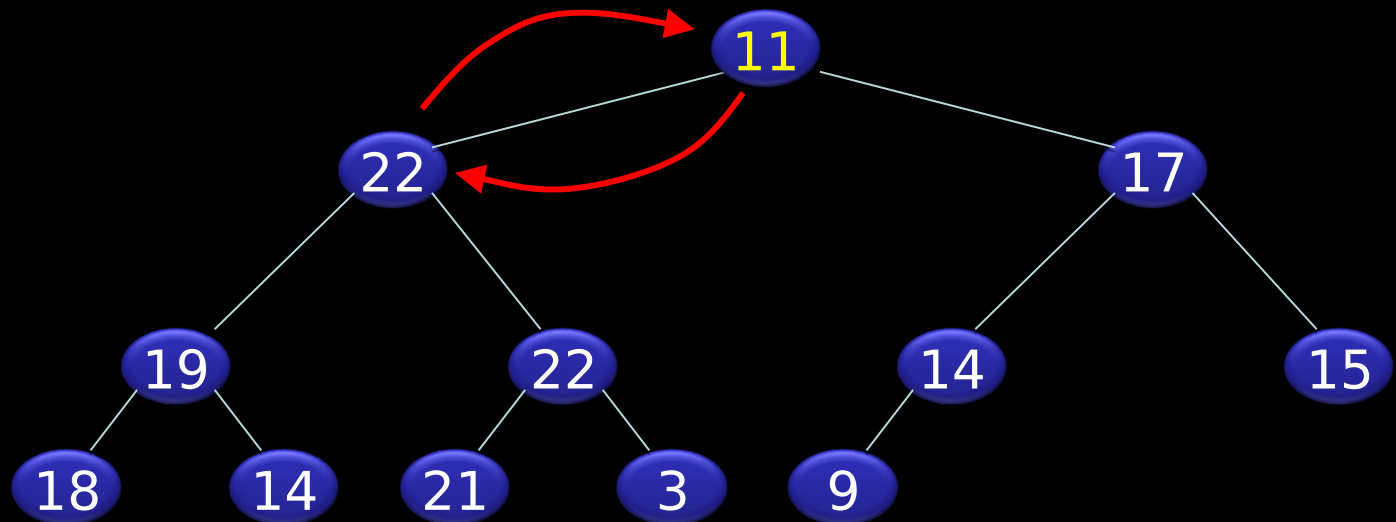
- Observe que o maior valor agora está na raiz
- Suponha que descartemos a raiz:



- Como consertamos a árvore para voltar a estar balanceada e justificada à esquerda?
- Solução: Retire o elemento mais à direita do nível mais inferior e use como raiz.

O método **reamontoar** I

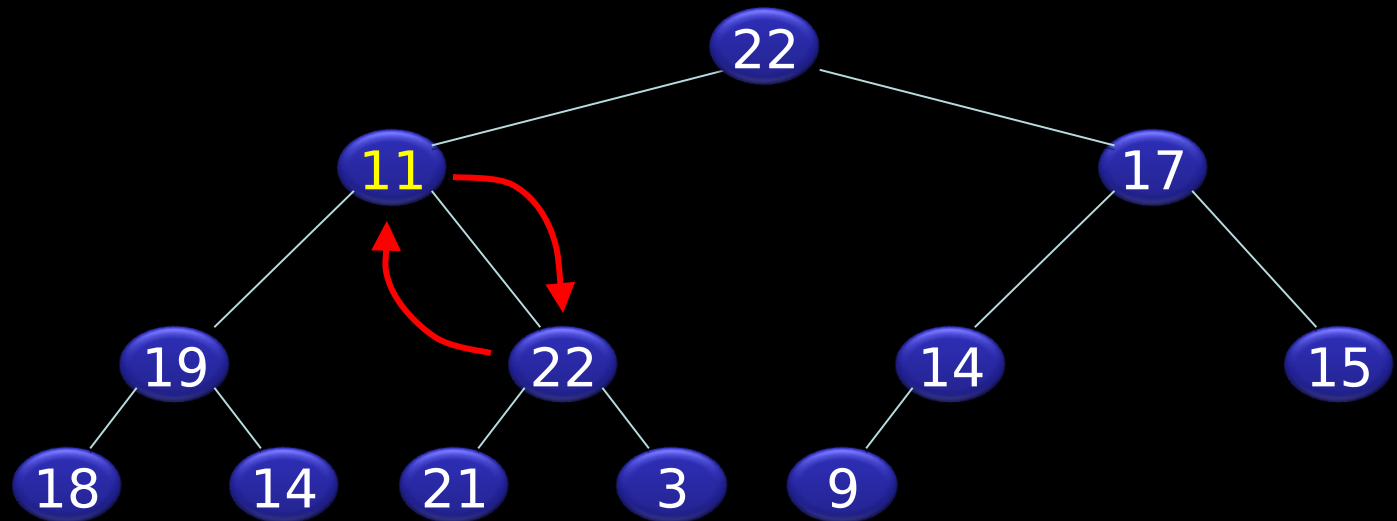
- A árvore está balanceada e justificada à esquerda, mas não é mais um **heap**
- Porém, *somente a raiz* não possui a **propriedade-heap**



- **Peneiramos para cima** a raiz (**sift up**)
- Feito isto, somente um filho perdeu a propriedade-heap

O método **reamontoar** II

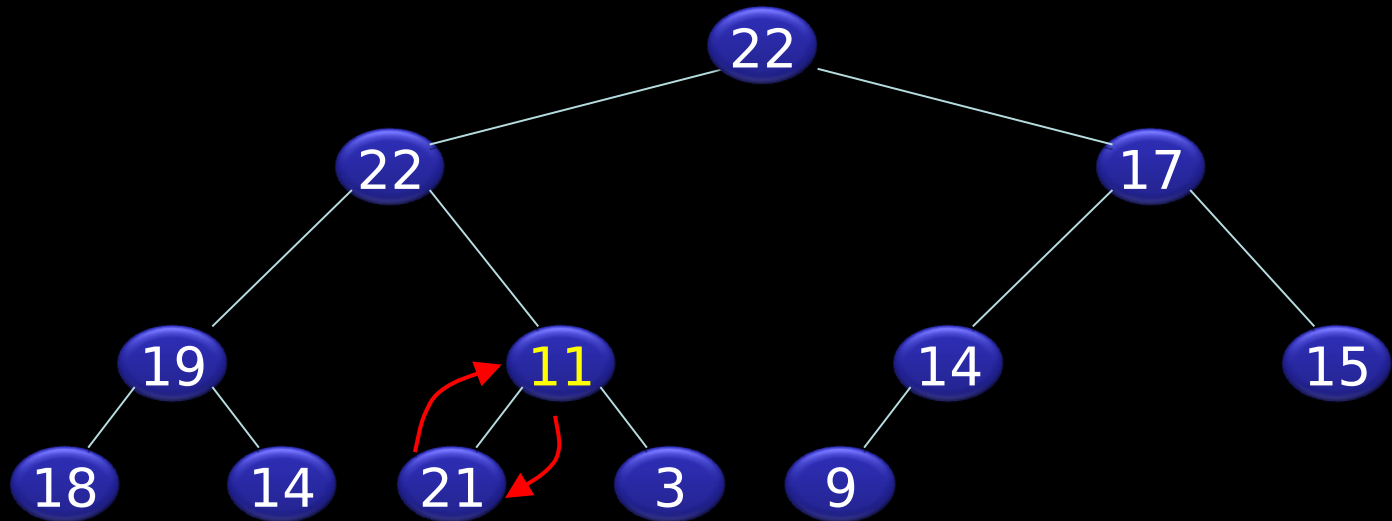
- Agora o filho à esquerda da raiz perdeu a propriedade-heap (também um 22)



- Este nodo podemos também **peneirar para cima**
- Novamente somente um filho perdeu a condição-heap

O método **reamontoar** III

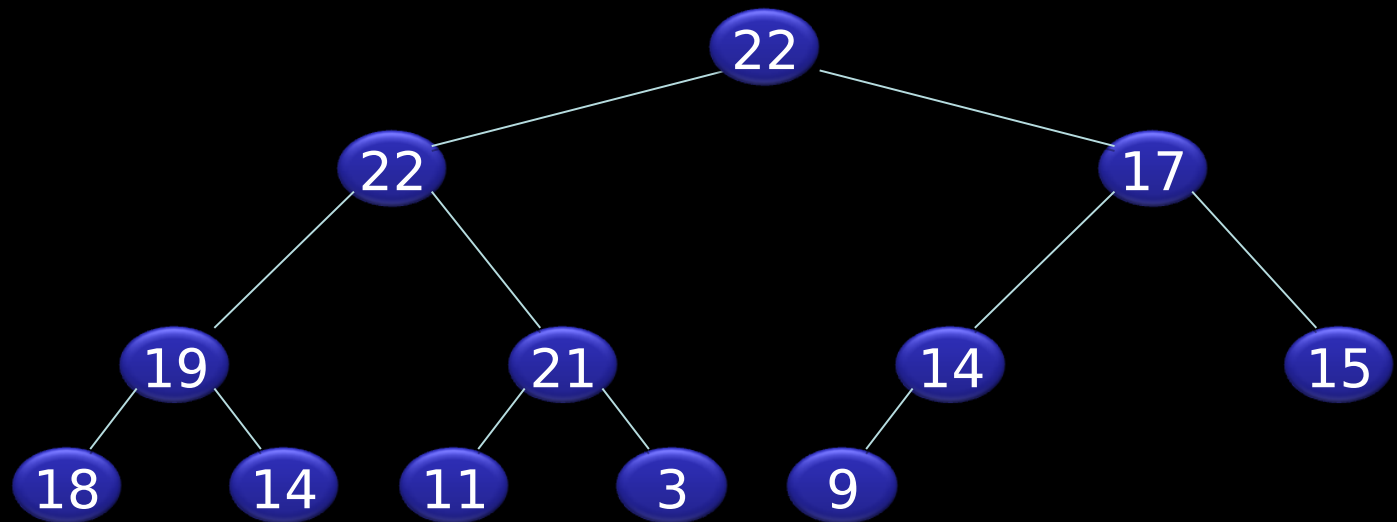
- Agora o filho à direita do filho à esquerda da raiz (**11**) é não-heap:



- Podemos **peneirar para cima** este nodo
- Depois, somente um de seus filhos poderia ter perdido a propriedade-heap, mas todos são folha!

O método **reamontoar** IV

- A árvore torna a ser um monte porque todos os nodos possuem a propriedade-heap



- O maior valor voltou a estar na raiz
- Podemos repetir isto até a árvore estar vazia
- Assim retiramos do maior ao menor